智慧居家安全監控系統

邊緣運算 + 即時監控+ 隱私保護

GROUP 2 | B 小町

N28141092
顏瑋良

N26130516
盧欣靖

E24114752
劉耘僑

E24114231
蕭汎穎

E24116330
嚴詠愉

專案動機 [MOTIVATION]

⚠ 傳統痛點 (Threats)

> **被動錄影：** 事件後才回放，缺乏即時性。

> **隱私疑慮：** 影像上雲端，外洩風險高。

> **人力短缺：** 少子化導致全時監控困難。

🛡 解決方案 (Solutions)

> **主動通知：** 偵測到「逗留」即刻發送警報。

> **邊緣運算：** 資料本地處理，隱私不外流。

> **低成本：** 使用 Raspberry Pi，無需昂貴主機。

⚠ 初版挑戰 (Feedback)

> **使用體驗：**警報過於頻繁 (誤報困擾) 。

> **介面呈現：**GUI 介面也會影響使用者體驗 。

🛡 解決方案 (Solutions)

> **新增調整功能：** 讓使用者依習慣調整信心度閥值。

> **修改架構：**捨棄傳統 GUI，改採 Web 架構 。

系統架構 [SYSTEM ARCHITECTURE]

影像輸入
USB Camera

邊緣推論
Raspberry Pi 4
YOLO

行為判斷
Logic & Filtering

即時監控
Streamlit / Email

隱私強化：馬賽克處理

模型輕量化：剪枝 (Pruning)
模型輕量化：量化 (Int8)
效能優化：辨識種類

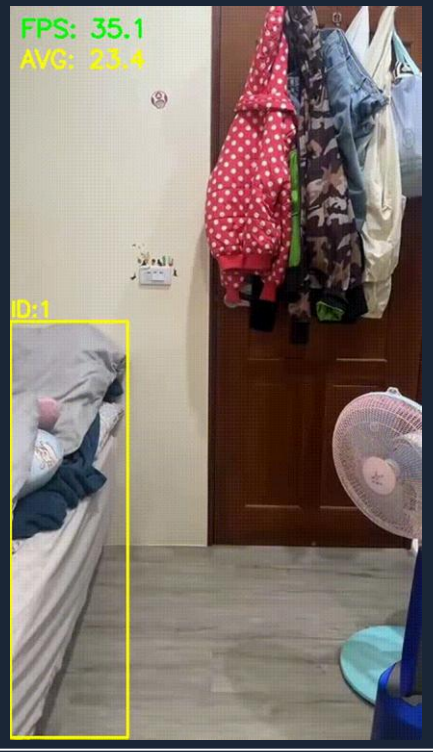
判斷升級：IoU 追蹤與狀態

系統彈性：Web 儀表板整合
主動通報：Email 警報推播

系統介面優化[From Desktop GUI to Web]

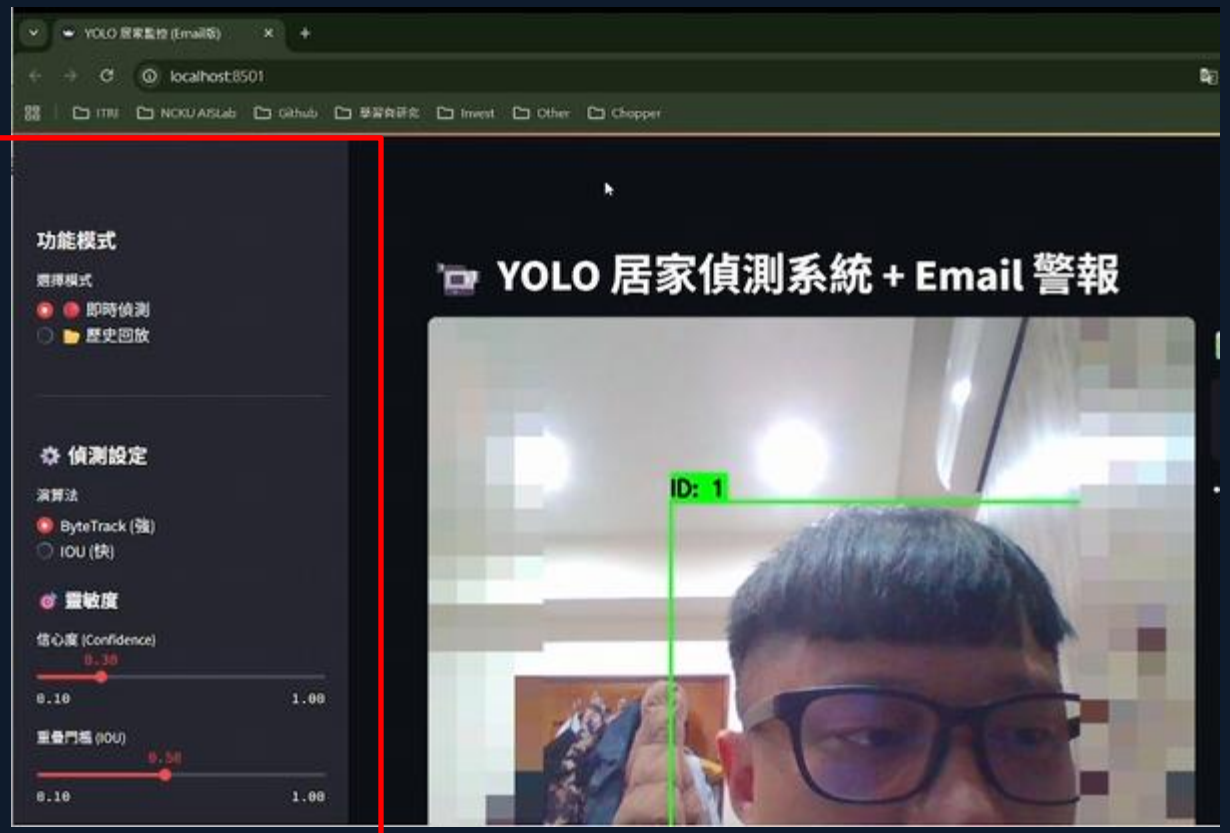
比較項目	方法 A：Desktop GUI + 遠端桌面	方法 B：Web (Streamlit) + 瀏覽器
顯示狀況	整個樹莓派桌面的錄影	網頁呈現與數據
手機操作性	差 (需放大縮小、模擬滑鼠)	優 (原生觸控、版面自動適配)
畫面清晰度	受網路影響，易有馬賽克	介面永遠清晰，僅影像區塊變動
樹莓派負載 (使用者體驗)	重 (需額外跑桌面遠端程式 + 影像編碼)	中 (需跑 Web Server)

辨識追蹤優化[OPTIMIZATION]



比較項目	方案 1	方案 2	方案 3	方案 4
模型策略	預設模型 (80類)	預設模型 (80類)	自訓練 (單一類別)	自訓練 (單一類別)
追蹤演算法	IoU (幾何計算)	ByteTrack (卡爾曼濾波)	IoU (幾何計算)	ByteTrack (卡爾曼濾波)
平均 FPS	29.95	27.62	30.75	29.36
綜合評價	速度優異，但包含冗餘類別	運算最重，FPS 最低	極致輕量化，適合單純場景	速度與抗遮擋穩定性最佳

警報疲勞優化 [OPTIMIZATION]



參數動態調整

🟦 信心度 (CONFIDENCE) [準度優先 vs. 靈敏優先]
開放使用者調整 YOLO 的判讀門檻。

- 討厭誤報者：可調高門檻，過濾掉誤判。
- 高度戒備者：可調低門檻，確保任何角落的人影都不漏接。

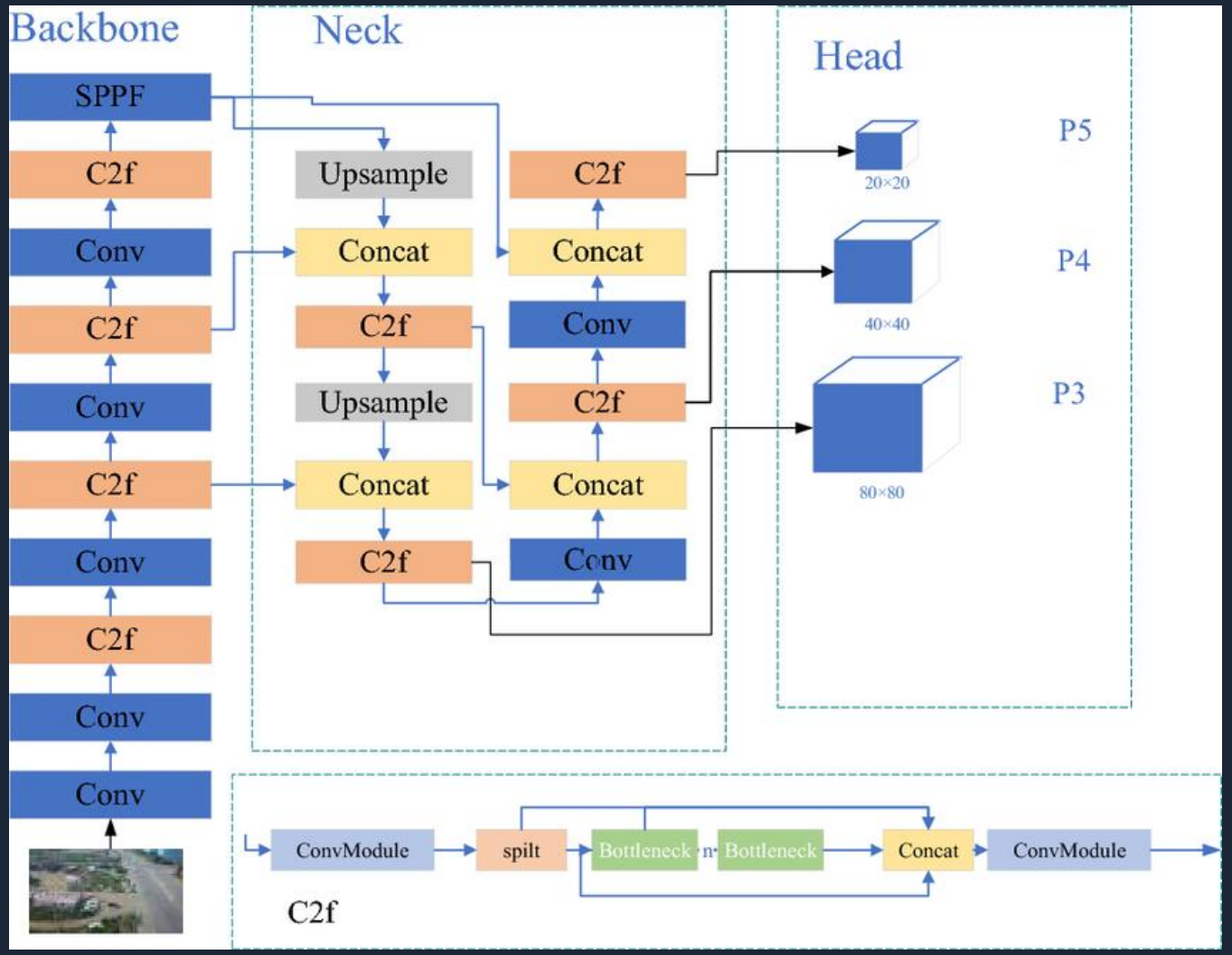
🟪 重疊門檻 (IOU THRESHOLD) [追蹤穩定度]
調整 IoU (Intersection over Union) 判定比例。

- 針對不同場域（如狹窄走廊或開闊客廳）調整追蹤邏輯，優化系統在不同距離下的判斷能力。

模型優化流程 [方法整理]

1. No-P3 架構
2. GhostNet
3. Torch_prune 50%通道剪枝
4. Multi step 50%通道剪枝

模型優化流程 [No-P3 架構]



場景：
家門口、玄關等等，人物通常偏大

No-P3 架構：
輸入尺寸 640x640
最小穩定高度 32 像素 (2個 grid)
佔比計算：32 / 640 = 5%

No-P3 運算量：
對運算量影響巨大，理論上減少 33%

模型優化流程 [No-P3 架構]

流程：

A
Dataset
Preparation

B
No-P3
Training

Colab 測試：

	mAP50	mAP50 降低	FLOPs(G)	FLOPs 降低	FPS	FPS 提升	Parameters(G)	Parameters 變化
yolo8vn	0.995	0	8.2	0	3.7	0	3.01	0
No-P3	0.9720	2.31%	6.0	26.83%	4.8	29.09%	3.1	-3.09%

模型優化流程 [GhostNet]

Idea

論文指出很多 CNN 的 feature map 間是相似的(視為線性相依)，因此可以取一部分 channel 做一般 conv 產生 Intrinsic Feature Maps ，另一部分用 Intrinsic Feature Maps 做計算量較低的 depthwise conv 產生 Ghost Feature Maps，藉此降低 FLOPS

Ghostconv(c1, c2)：

輸出通道 c2：GhostConv 的最終輸出通道數
intrinsic 通道數：ch = c2/2
cv1：Conv(c1 → ch, k×k, groups=g) 產生 intrinsic maps：y
cv2：Depthwise Conv(ch→ch, 5×5, groups=ch) 產生 ghost maps：g
concat：z = cat(y, g) → 輸出通道 ch+ch = c2

模型優化流程 [GhostNet]

Ghostconv：

CV1 Parameters = $k^2 \times C1 \times Ch + 2Ch$

CV2 Parameters = $5^2 \times Ch + 2Ch = 27Ch$

CV1 + CV2 Parameters = $\boxed{k^2 \times C1 \times Ch} + 29Ch$

MACS = $\boxed{H \times W \times k^2 \times C1 \times Ch} + H \times W \times 29Ch$

VS

標準conv：

Parameters = $\boxed{k^2 \times C1 \times C2(2Ch)} + 2C2(4Ch)$

MACS = $\boxed{H \times W \times k^2 \times C1 \times C2(2Ch)}$

```
Non-Ghost Baseline:
  path   : /content/yolo_optimization/yolov8n_baseline_320/weights/best.pt
  params: 3,011,238
  flops : 1.02 GFLOPs

Ghost Baseline:
  path   : /content/yolo_optimization/ghost_baseline_320/weights/best.pt
  params: 1,719,354
  flops : 0.64 GFLOPs
```

模型優化流程 [GhostNet]

A Non-Ghost Baseline → **B Ghost Backbone** → C Sparse Training → D Width Scaling → E Finetune

A Non-Ghost Baseline：YOLOv8n 原版架構（Conv/C2f）作為對照組

B Ghost Baseline：用 Ghost module 降低 FLOPS

C Sparse Training：在訓練時對 BatchNorm 的 γ（weight） 加入 L1 稀疏化

D Width Scaling： 直接縮小每層通道數

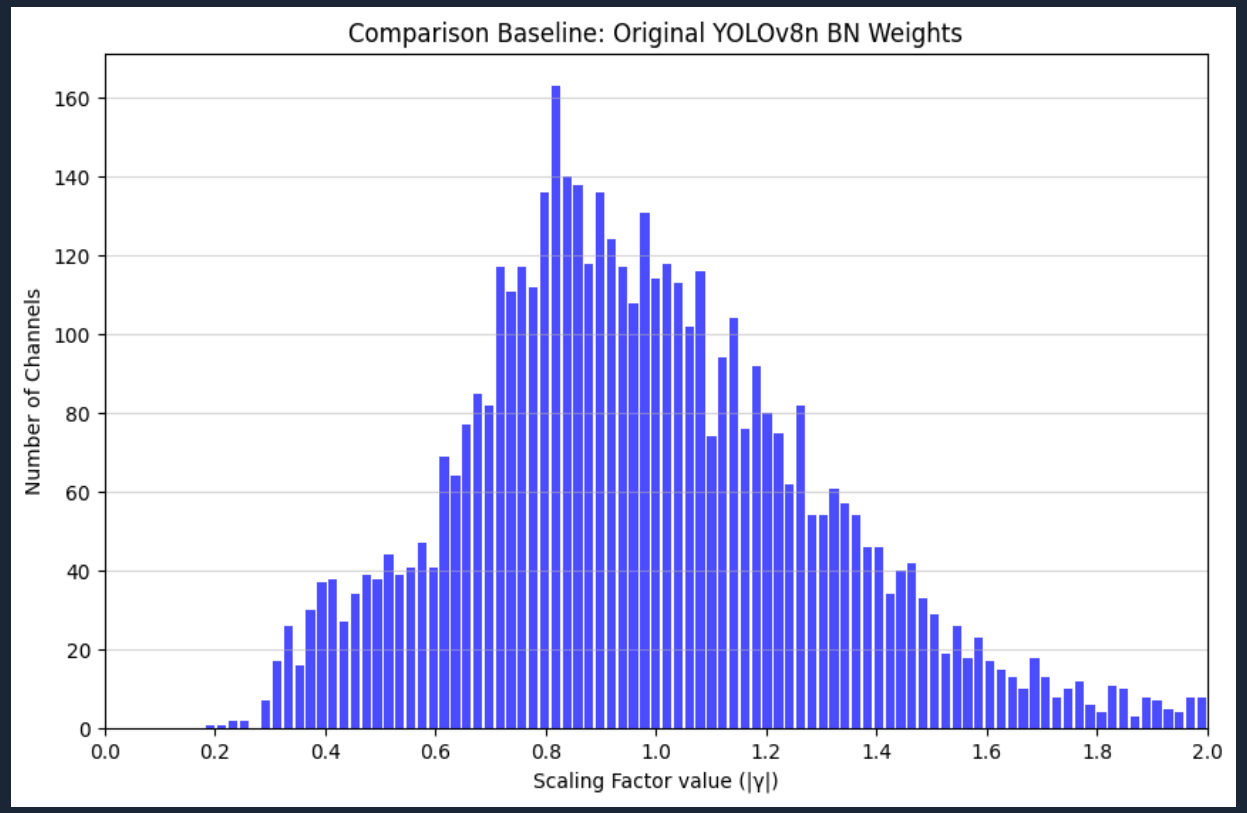
E Finetune：微調恢復精度

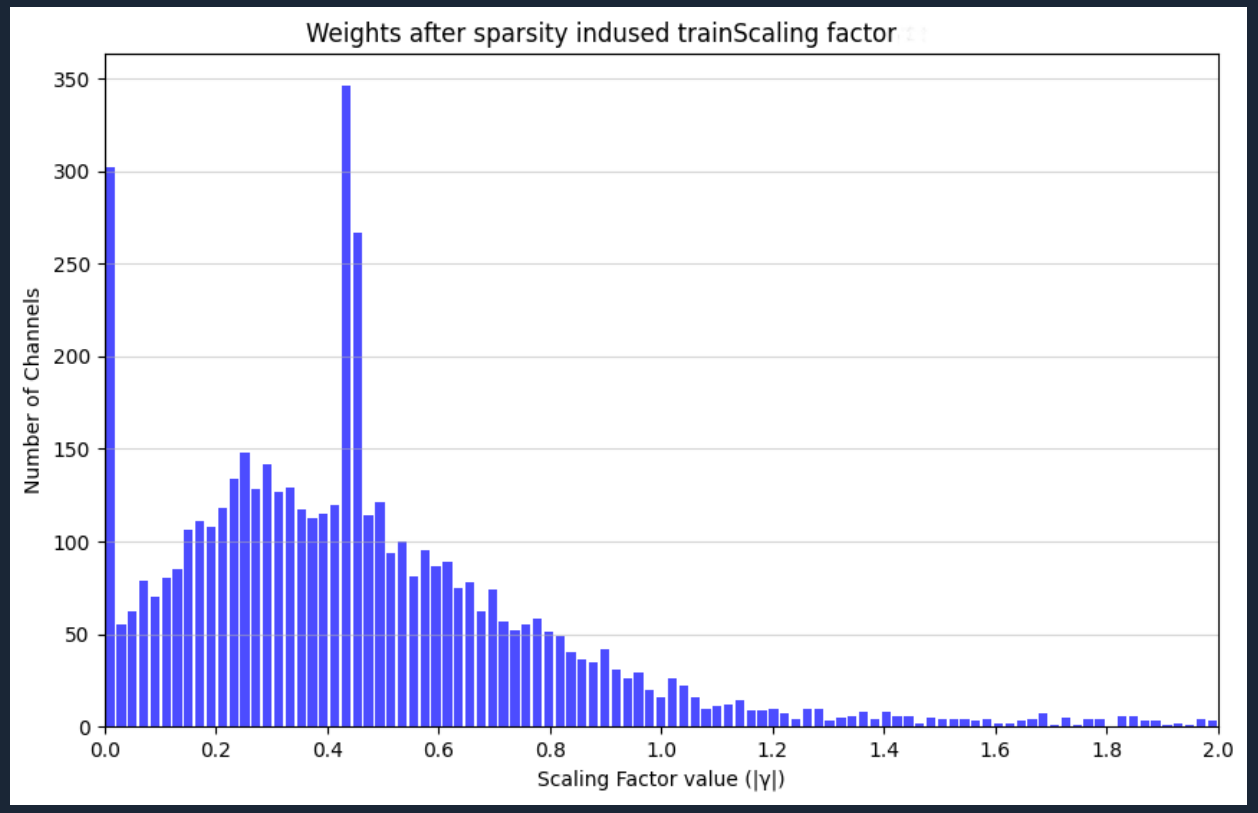
模型優化流程 [GhostNet]

	mAP50	mAP50 損失	FLOPs (G)	FLOPs 減少	FPS	FPS提升	參數量 (MB)	參數量 減少
yolo8vn Baseline	0.7398	0	1.02	0	24.9	0	3.01	0
Ghost Baseline	0.6297	-14.88%	0.64	-37.25%	30.72	+23.37%	1.72	-42.86%
Ghost Sparse	0.6827	-7.72%	0.64	-37.25%	26.38	+5.94%	1.72	-42.86%
Ghost width×0.7 + finetune	0.7396	約 = 0	0.43	-57.84%	37.95	+52.41%	1.04	-65.45%

模型優化流程 [Torch_prune 50%通道剪枝]

1. Sparsity Induced Train





模型優化流程 [Torch_prune 50%通道剪枝]

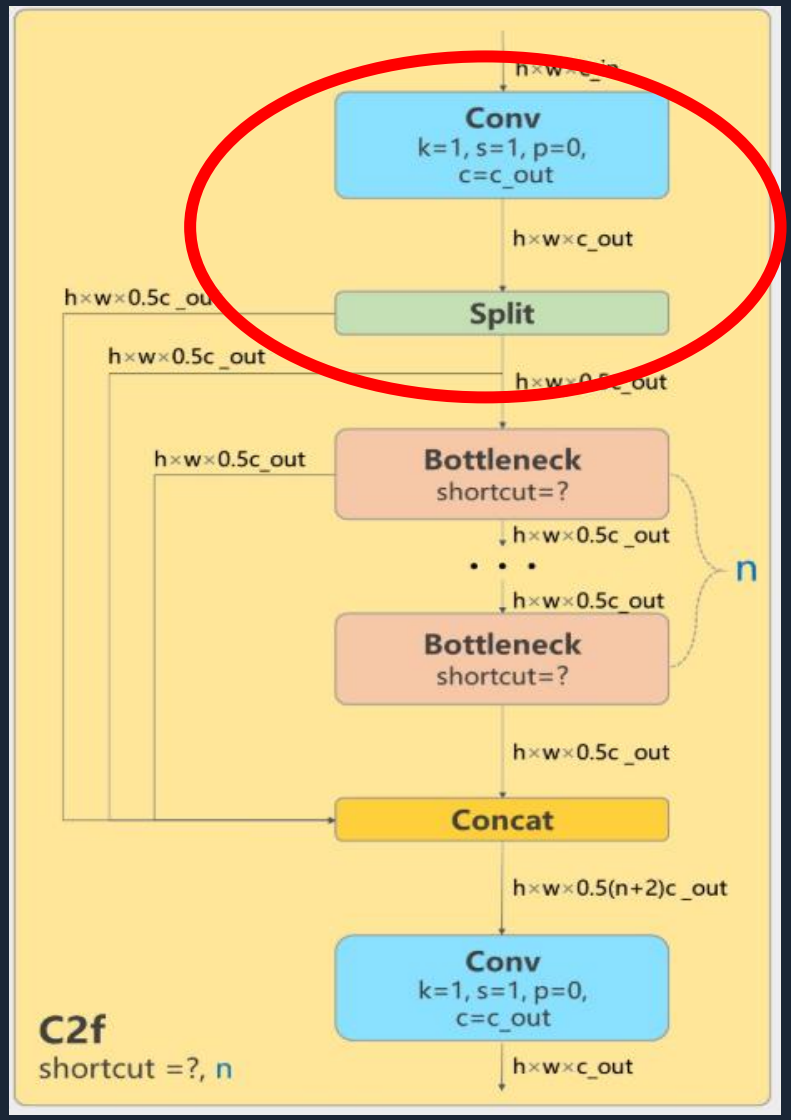
2. Decouple C2f conv1

- 原版:

  Conv1 output 依通道split成兩個分支。

- 修改:

  直接用兩個分開的 conv，
  輸出通道數各為原 conv1 的一半。

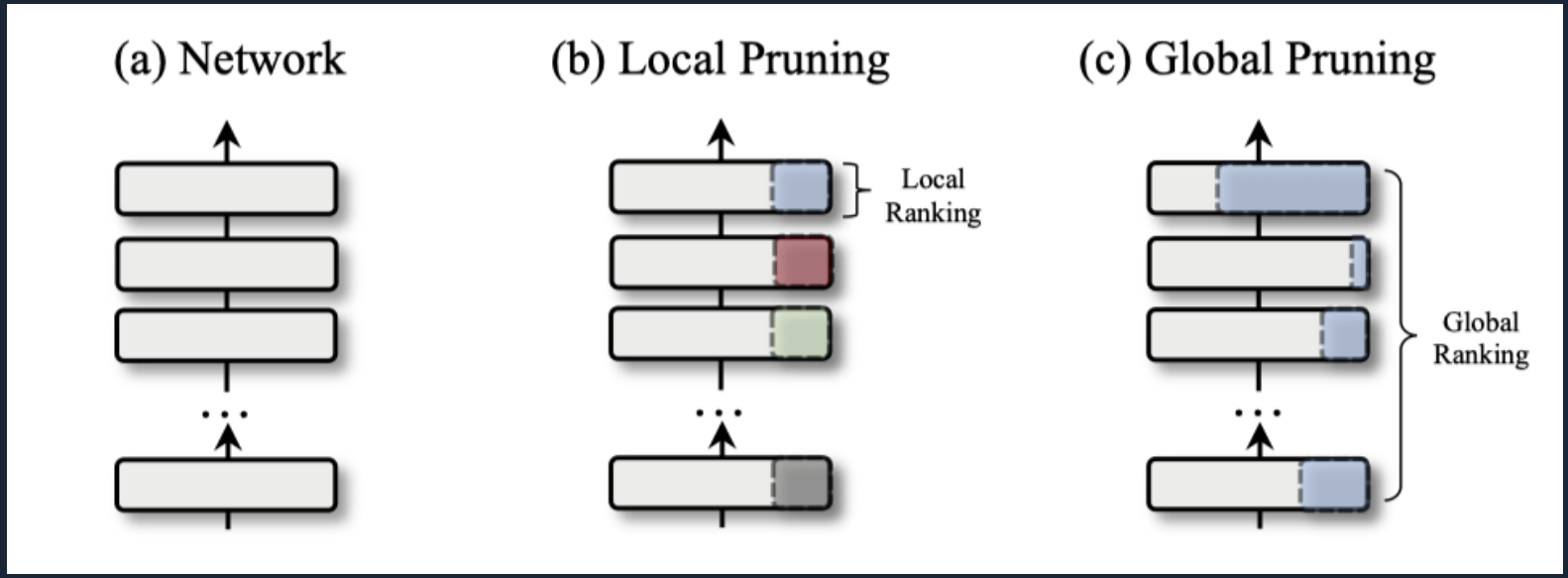


模型優化流程 [Torch_prune 50%通道剪枝]

3. 通道剪枝

- Importance： Group Magnitude Importance (p=1, target = BN)

- Global_pruning = True



模型優化流程 [Torch_prune 50%通道剪枝]

3. 通道剪枝

- Pruning_ratio = 0.5
  剪掉50%的通道 =>
  參數量會剪掉 1 - (1 - 0.5)^2 = 75%

- Iterative_steps = 1

- Ignored_layers = Detect

模型優化流程 [Torch_prune 50%通道剪枝]

	Parameters(k)	FLOP(G)	P	mAP50
原模型	3151.9	8.7	0.817	0.801
50% 通道剪枝	1046.6	3.5	0.687	0.601

模型優化流程 [Multi step 50%通道剪枝]

Step 1
- 2404.2 k parameters
- 6.8 GFLOPs
- Train 5 epochs

Step 2
- 1900.9 k parameters
- 5.7 GFLOPs
- Train 7 epochs

Step 3
- 1,454.0 k parameters
- 4.7 GFLOPs
- Train 10 epochs

Step 4
- 1049.4 k parameters
- 3.9 GFLOPs
- Train 30 epochs

模型優化流程 [Multi step 50%通道剪枝]

	Parameters(k)	FLOP(G)	P	mAP50
原模型	3151.9	8.7	0.817	0.801
50% 通道剪枝	1046.6	3.5	0.687	0.601
50%, 4 step	1049.4	3.9	0.774	0.682

模型優化流程 [Multi step 50%通道剪枝]

	P (60 epochs)	mAP50 (fine tuned)
原模型	0.817	0.801
50%, 4 step	0.802	0.702
Int 8 (.mnn)	0.814	0.698

模型優化流程 [Multi step 50%通道剪枝]

	Fps, on colab CPU
Yolov8n.pt	12.37
50%, 1 step (.pt)	20.82
50%, 4 step (.pt)	21.65
Yolov8n.mnn	12.11
FP32.mnn	20.95
Int 8 (.mnn)	20.93

模型優化流程 [樹梅派實際測試]

```
================================================================
Comparison Results
================================================================
Model                                Avg(ms)      FPS
----------------------------------------------------------------
v2_fine_after_prune50_4step_fp32mnn 115.83       8.63
v2_fine_after_prune50_4step_int8mnn 115.90       8.63
v2_fine_after_prune50_4step_ncnn 124.53       8.03
v2_fine_after_prune50_4step_onnx 144.76       6.91
v1_fine_after_prune_int8_mnn    170.64       5.86
v1_fine_after_prune_fp32_mnn    170.89       5.85
v1_fine_after_prune_ncnn        176.28       5.67
v1_fine_after_prune_onnx        207.15       4.83
v2_fine_after_prune50_4step_pt 259.00       3.86
v1_fine_after_prune_pt          340.07       2.94
v1_pre_train_pt                 388.28       2.58
nop3_yolo_model_pruned_10_pt    391.14       2.56
nop3_yolo_baseline_nop3_person_pt 391.84       2.55
nop3_yolo_model_pruned_30_pt    399.56       2.50
nop3_yolo_model_pruned_50_pt    403.58       2.48
yolov8n_official                424.60       2.36
```

模型優化流程 [最終選擇模型與演算法策略]

```
================================================================
Comparison Results
================================================================
Model                                Avg(ms)      FPS
----------------------------------------------------------------
v2_fine_after_prune50_4step_fp32mnn 115.83        8.63
v2_fine_after_prune50_4step_int8mnn 115.90        8.63
v2_fine_after_prune50_4step_ncnn 124.53        8.03
v2_fine_after_prune50_4step_onnx 144.76        6.91
v1_fine_after_prune_int8_mnn    170.64       5.86
v1_fine_after_prune_fp32_mnn    170.89       5.85
v1_fine_after_prune_ncnn        176.28       5.67
v1_fine_after_prune_onnx        207.15       4.83
v2_fine_after_prune50_4step_pt 259.00       3.86
v1_fine_after_prune_pt          340.07       2.94
v1_pre_train_pt                 388.28       2.58
nop3_yolo_model_pruned_10_pt    391.14       2.56
nop3_yolo_baseline_nop3_person_pt 391.84       2.55
nop3_yolo_model_pruned_30_pt    399.56       2.50
nop3_yolo_model_pruned_50_pt    403.58       2.48
yolov8n_official                424.60       2.36
```

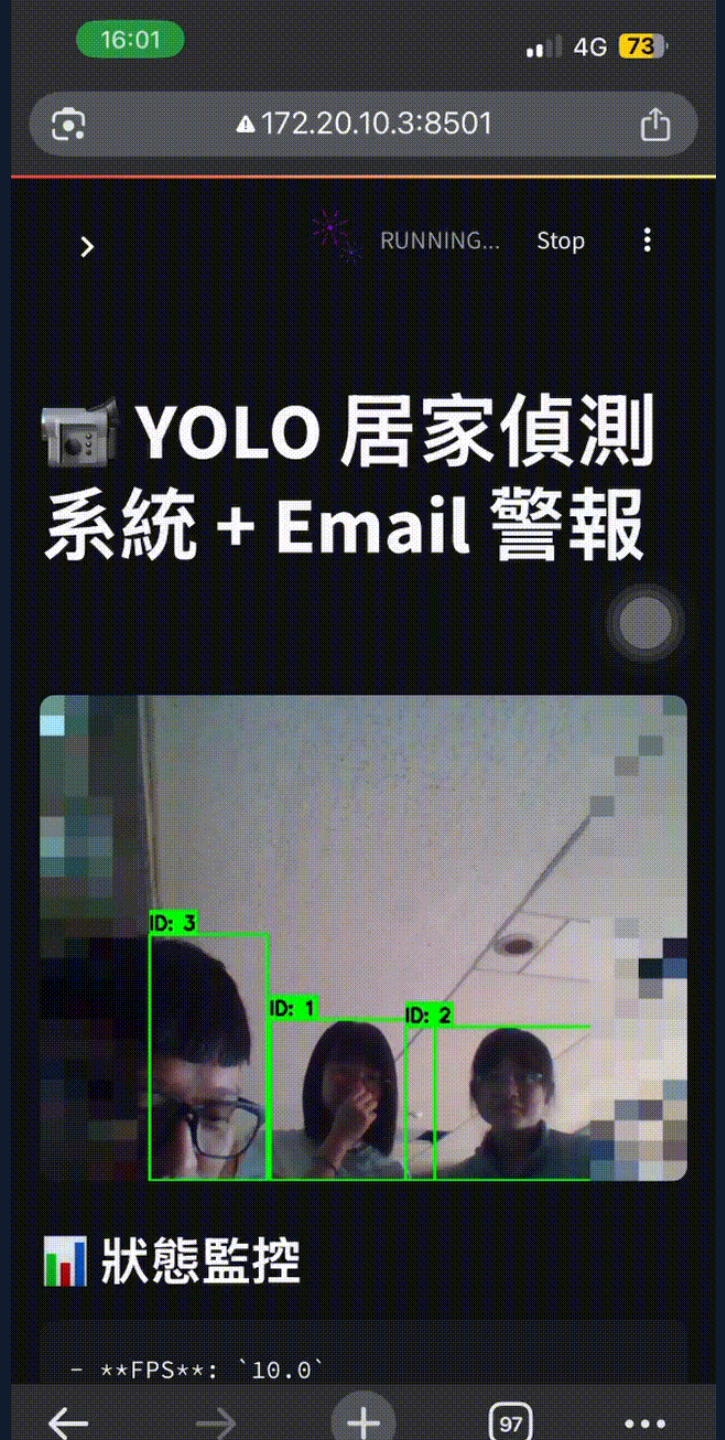
演算法策略：
可在介面中切換
IOU 或 ByteTrack

實際演示 [DEMO]

01 // 正常活動)
動作：快速通過鏡頭 → 無警報

02 // 系統參數調整 (CONFIG)
動作：透過 Web UI 調整信心度、馬賽克、Email通知…等

03 //可疑人物
動作：逗留超過 10 秒 → 觸發警報 & 發送截圖至Email



Reference

https://github.com/ultralytics/ultralytics/issues/189

- https://github.com/VainF/Torch-Pruning
- https://blog.csdn.net/a486259/article/details/140421837
- Han, K., Wang, Y., Tian, Q., Guo, J., Xu, C., & Xu, C. “GhostNet: More Features From Cheap Operations,” CVPR, 2020
- https://miro.medium.com/1*k7EqcIYh7q4c6tx__yljyg.png
- https://cdn.dribbble.com/userupload/44075575/file/original-c347f3abbb947b6bf51671f43f83f8c7.webp?resize=752x&vertical=center
- https://b2633864.smushcdn.com/2633864/wp-content/uploads/2016/01/multiple_cameras_rpi_start.jpg?lossy=2&strip=1&webp=1

Q & A

SESSION TERMINATED